PE#5

Start Assignment

Due Monday by 11:59pm **Points** 10 **Submitting** a text entry box or a file upload
Available until Jul 13 at 11:59pm

In-Class Practice (2 hours)

These are progressively harder. Pair-programming encouraged. Use your own Week 4 framework as the starting point.

Q1 (Easy) — Install and Verify

Install `xlsx` and `csv-parse` in your Week 4 framework. Run `npm ls xlsx csv-parse` and paste the output into a comment in your project's README. Both packages should appear without warnings.

Q2 (Easy) — A Tiny Spreadsheet

Create `data/practice.xlsx` with these 5 rows:

id	name	age
1	Alice	30
2	Bob	25
3	Carol	45
4	Dan	19
5	Eve	99

Write a script `scripts/dump-practice.js` that reads the file with `xlsx`, prints the rows, and prints the total count. No Selenium, no Mocha — just verify your reader works.

Q3 (Easy) — A Tiny CSV

Create `data/practice.csv` with the **same** 5 rows. Modify `scripts/dump-practice.js` to also print the rows from the CSV using `csv-parse`. Confirm both outputs match exactly.

Q4 (Easy) — Build the `readExcel` Helper

Create `utils/excel.js` with a `readExcel(filepath, sheetName?)` function as shown in Section 2. Add an optional second parameter for sheet name. Throw a clear error if the sheet doesn't exist. Export it.

Q5 (Medium) — Build the `readCsv` Helper

Create `utils/csv.js` with a `readCsv(filepath)` function using `csv-parse`. Include `bom: true` and `trim: true`. Export it. Use it in `scripts/dump-practice.js` instead of inlining `parse(...)`.

Q6 (Medium) — Data-Driven Login from CSV

Take ONE existing test from your framework (login is easiest). Create `data/login-data.csv` with at least **10 rows** including:

- 2 valid combinations
- 3 invalid combinations
- 1 empty-username case
- 1 empty-password case
- 1 SQL-injection-style string
- 1 unicode/special-character case
- 1 locked-out user (if your target supports it)

Write `tests/login.dd.spec.js` that uses `readCsv` and the `forEach`/`it` pattern.
Add `testId` and `testCase` to each row. Run it. All 10 should pass (because the assertions match the `expected` column).

Q7 (Medium) — Verify the Report

Run `npm test` and open the mochawesome report. Confirm:

- All 10 rows appear as separate test entries
- Each test name starts with `[testId]` (e.g., `[L01]`)
- Test descriptions are meaningful at a glance
- Failures (if you intentionally break one row) point to the right `testId`

To intentionally break one: change `expected` for `L02` from `pass` to `fail`. Re-run. Confirm the report flags row `L02` as failing with a useful message.

Q8 (Medium) — Switch to Excel

Convert your Q6 CSV to an Excel file (`data/login-data.xlsx`). Update `tests/login.dd.spec.js` to use `readExcel` instead of `readCsv`. Run again. Same 10 tests should pass.

You now have **two** data sources for the same logical test. In `tests/login.dd.spec.js`, leave a comment: `// Source: data/login-data.xlsx (mirror in login-data.csv)`.

Q9 (Harder) — Two-Sheet Spreadsheet

Open `data/login-data.xlsx` and add a second sheet named `EdgeCases` with 5 weird rows (very long input, leading/trailing spaces, embedded newline, emoji, control characters). Modify your test to run BOTH sheets:

```
const main = readExcel("data/login-data.xlsx");
const edge = readExcel("data/login-data.xlsx", "EdgeCases");

describe("Login - Main", function () { main.forEach(row => it(...)); });
describe("Login - Edge Cases", function () { edge.forEach(row => it(...)); });
```

Run. Verify both describe blocks appear in the mochawesome report.

Q10 (Harder / Discussion)

In a 2-3 sentence comment in your test file, answer:

> "What kinds of test data should NOT live in `/data` (and where should they live instead)?"

Examples to think about: real customer credentials, API tokens, environment-specific URLs, generated random emails, ephemeral test users created during the run.

Compare with your pair. Your instructor will discuss correct answers during wrap-up.
